

Lập trình web

LẬP TRÌNH VỚI ASP.NET CORE MVC P4

2

Nội dung

1. Giới thiệu về Dependency Injection
2. Lập trình Bất đồng bộ trong C#
2. Khởi tạo dự án thương mại điện tử
3. Authentication và Authorization

GIỚI THIỆU VỀ DEPENDENCY INJECTION

4

Dependency Injection (DI) là gì?

```
public class StudentService
{
    private readonly StudentRepository _repository = new StudentRepository();

    public void DoSomething()
    {
        _repository.GetStudents();
    }
}
```

5

Dependency Injection (DI) là gì?

- **Dependency Injection** là một kỹ thuật trong lập trình dùng để **giảm sự phụ thuộc giữa các lớp** (loose coupling) bằng cách **tiêm đối tượng phụ thuộc từ bên ngoài** thay vì tự tạo bên trong lớp.

```
public class StudentService
{
    private readonly IStudentRepository _repository;

    public StudentService(IStudentRepository repository)
    {
        _repository = repository;
    }

    public void DoSomething()
    {
        _repository.GetStudents();
    }
}
```

6

DI hoạt động thế nào trong ASP.NET Core?

- ASP.NET Core tích hợp sẵn DI container.
- Bạn chỉ cần: Đăng ký dịch vụ (service) trong Program.cs hoặc Startup.cs
- Sử dụng (inject) dịch vụ ở nơi cần thiết (Controller, Service, v.v.)

```
public interface IMessageService
{
    string GetMessage();
}
```

```
public class HelloMessageService : IMessageService
{
    public string GetMessage()
    {
        return "Xin chào từ DI!";
    }
}
```

```
builder.Services.AddScoped<IMessageService, HelloMessageService>();
```

7

DI hoạt động thế nào trong ASP.NET Core?

```
public class HomeController : Controller
{
    private readonly IMessageService _messageService;

    public HomeController(IMessageService messageService)
    {
        _messageService = messageService;
    }

    public IActionResult Index()
    {
        ViewBag.Message = _messageService.GetMessage();
        return View();
    }
}
```

8

Các phạm vi dịch vụ trong ASP.NET Core

Phạm vi	Ý nghĩa
Singleton	Tạo một lần duy nhất cho toàn ứng dụng
Scoped	Tạo một lần cho mỗi request
Transient	Tạo mới mỗi lần được gọi

LẬP TRÌNH BẤT ĐỒNG BỘ TRONG C#

Lập trình bất đồng bộ là gì?

📁 Đồng bộ (Synchronous):

- Thực hiện tuần tự từng dòng lệnh.
- Mỗi lệnh phải hoàn tất mới đến lệnh kế tiếp.
- Nếu một thao tác mất thời gian (I/O, mạng, DB), ứng dụng sẽ chờ.

👉 Kết quả: **treo ứng dụng**, chậm phản hồi (đặc biệt với UI hoặc Web).

⚡ Bất đồng bộ (Asynchronous):

- Cho phép chương trình tiếp tục thực hiện các tác vụ khác trong khi chờ tác vụ dài (I/O, DB...) hoàn thành.
- Giúp ứng dụng phản hồi tốt hơn, tận dụng tài nguyên hiệu quả hơn.

Từ khóa async và await trong C#

💡 Nguyên tắc:

- `async` dùng để khai báo phương thức bất đồng bộ.
- `await` dùng để đợi kết quả của một phương thức bất đồng bộ khác (và không chặn luồng chính).

```
public async Task<string> GetDataAsync()
{
    await Task.Delay(3000); // giả lập chờ 3 giây
    return "Dữ liệu đã sẵn sàng!";
}
```

Cách dùng trong ứng dụng

➤ Gọi từ một controller trong ASP.NET Core:

```
public class HomeController : Controller
{
    public async Task<IActionResult> Index()
    {
        string data = await GetDataAsync();
        ViewBag.Message = data;
        return View();
    }

    private async Task<string> GetDataAsync()
    {
        await Task.Delay(1000); // mô phỏng độ trễ
        return "Xin chào từ bất đồng bộ!";
    }
}
```

13

So sánh Sync vs Async

Đặc điểm	Synchronous	Asynchronous
Giao diện phản hồi	Chậm nếu có tác vụ nặng	Mượt mà và không bị chặn
Tận dụng CPU	Kém	Hiệu quả hơn
Dễ viết	Có	Cần hiểu <code>Task</code> , <code>async/await</code>
Phù hợp	Tác vụ nhỏ, nhanh	I/O-bound (file, DB, API call)

14

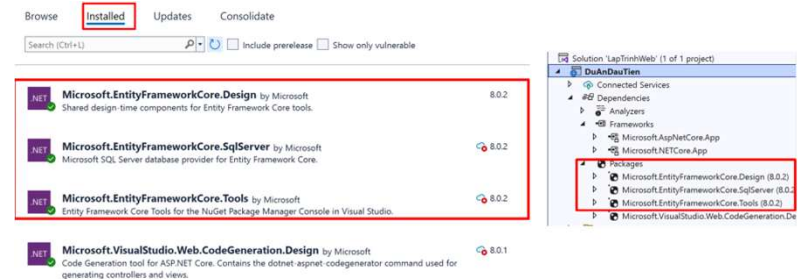
Ứng dụng phổ biến của Async trong .NET

- Gọi Web API: `await httpClient.GetAsync(...)`
- Truy vấn DB EF Core: `await dbContext.Users.ToListAsync()`
- Đọc/Ghi file: `await File.ReadAllTextAsync(...)`
- Delay giả lập: `await Task.Delay(...)`

KHỞI TẠO DỰ ÁN THƯƠNG MẠI ĐIỆN TỬ

16

Bước 1: THIẾT LẬP CÁC GÓI NUGET CẦN THIẾT



17

BƯỚC 2: THIẾT KẾ MODEL

- Models
 - C# ApplicationDbContext.cs
 - C# Category.cs
 - C# ErrorViewModel.cs
 - C# Product.cs**
 - C# ProductImage.cs

```

Product.cs
public class Product
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    [DisplayFormat("Mã sản phẩm")]
    public int Id { get; set; }

    [Required(ErrorMessage = "Tên sản phẩm là bắt buộc"), StringLength(100, ErrorMessage = "Tên sản phẩm không được vượt quá 100 ký tự")]
    [DisplayFormat("Tên sản phẩm")]
    public string Name { get; set; }

    [Range(1.000, 100000.000, ErrorMessage = "Giá sản phẩm phải nằm trong khoảng từ 1.000 đến 100000.000")]
    [DisplayFormat("Giá bán")]
    public decimal Price { get; set; }

    [DisplayFormat("Mô tả")]
    public string Description { get; set; }

    [DisplayFormat("Hình ảnh")]
    public string? ImageUrl { get; set; }

    public List<ProductImage> Images { get; set; }

    [ForeignKey("Category")]
    [DisplayFormat("Mã danh mục")]
    public int CategoryId { get; set; }

    [DisplayFormat("Danh mục")]
    public Category? Category { get; set; }
}

```

18

BƯỚC 2: THIẾT KẾ MODEL

- Models
 - C# ApplicationDbContext.cs
 - C# Category.cs
 - C# ErrorViewModel.cs
 - C# Product.cs
 - C# ProductImage.cs**

```

ProductImage.cs
public class ProductImage
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    public string Url { get; set; }

    [ForeignKey("Product")]
    public int ProductId { get; set; }

    public Product? Product { get; set; }
}

```

19

BƯỚC 2: THIẾT KẾ MODEL

- Models
 - C# ApplicationDbContext.cs
 - C# Category.cs**
 - C# ErrorViewModel.cs
 - C# Product.cs
 - C# ProductImage.cs

```

Category.cs
public class Category
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required(ErrorMessage = "Tên danh mục là bắt buộc")]
    [StringLength(50, ErrorMessage = "Tên danh mục không được vượt quá 50 ký tự")]
    public string Name { get; set; }

    public List<Product> Products { get; set; }
}

```

20

BƯỚC 2: THIẾT KẾ MODEL

- Models
 - C# ApplicationDbContext.cs**
 - C# Category.cs
 - C# ErrorViewModel.cs
 - C# Product.cs
 - C# ProductImage.cs

```

ApplicationDbContext.cs
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
    base(options)
    {
    }

    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }
    public DbSet<ProductImage> ProductImages { get; set; }
}

```

21

BƯỚC 3: THIẾT KẾ REPOSITORY

- Repository
 - EFCategoryRepository.cs
 - EFProductRepository.cs
 - ICategoryRepository.cs
 - IProductRepository.cs**

```

IProductRepository.cs

public interface IProductRepository
{
    Task<IEnumerable<Product>> GetAllAsync();
    Task<Product> GetByIdAsync(int id);
    Task AddAsync(Product product);
    Task UpdateAsync(Product product);
    Task DeleteAsync(int id);
}

```

22

BƯỚC 3: THIẾT KẾ REPOSITORY

- Repository
 - EFCategoryRepository.cs
 - EFProductRepository.cs**
 - ICategoryRepository.cs
 - IProductRepository.cs

```

EFProductRepository.cs

public class EFProductRepository : IProductRepository
{
    private readonly ApplicationDbContext _context;

    public EFProductRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task<IEnumerable<Product>> GetAllAsync()
    {
        return await _context.Products.Include(x => x.Category).ToListAsync();
    }

    public async Task<Product> GetByIdAsync(int id)
    {
        return await _context.Products.Include(x => x.Category).SingleOrDefaultAsync(x =>
            x.Id == id);
    }

    public async Task AddAsync(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(Product product)
    {
        _context.Products.Update(product);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(int id)
    {
        var product = await _context.Products.FindAsync(id);
        _context.Products.Remove(product);
        await _context.SaveChangesAsync();
    }
}

```

23

BƯỚC 3: THIẾT KẾ REPOSITORY

- Repository
 - EFCategoryRepository.cs
 - EFProductRepository.cs
 - ICategoryRepository.cs**
 - IProductRepository.cs

```

ICategoryRepository.cs

public interface ICategoryRepository
{
    Task<IEnumerable<Category>> GetAllAsync();
    Task<Category> GetByIdAsync(int id);
    Task AddAsync(Category category);
    Task UpdateAsync(Category category);
    Task DeleteAsync(int id);
}

```

24

BƯỚC 3: THIẾT KẾ REPOSITORY

- Repository
 - EFCategoryRepository.cs**
 - EFProductRepository.cs
 - ICategoryRepository.cs
 - IProductRepository.cs

```

EFCategoryRepository.cs

public class EFCategoryRepository : ICategoryRepository
{
    private readonly ApplicationDbContext _context;

    public EFCategoryRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task<IEnumerable<Category>> GetAllAsync()
    {
        return await _context.Categories.ToListAsync();
    }

    //Làm tương tự như Product
}

```

25

BƯỚC 4: CẤU HÌNH EF CORE

```

Program.cs

using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

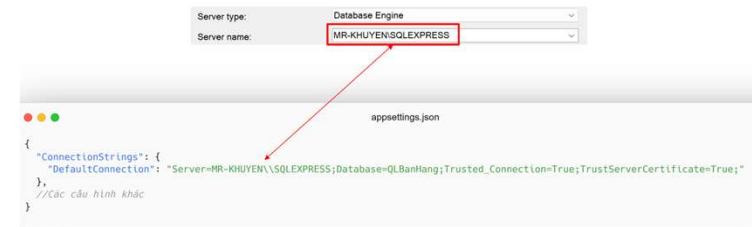
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<IProductRepository, EFProductRepository>();
builder.Services.AddScoped<ICategoryRepository, EFCategoryRepository>();

```

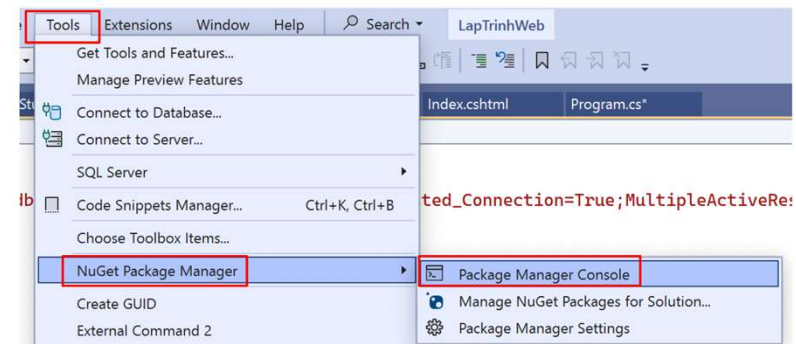
26

BƯỚC 4: CẤU HÌNH EF CORE



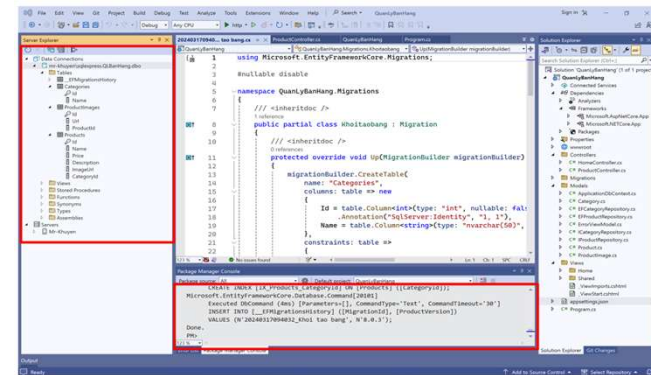
27

BƯỚC 5: TẠO VÀ THỰC THI MIGRATIONS



28

BƯỚC 5: TẠO VÀ THỰC THI MIGRATIONS



29

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

XEM DANH SÁCH
CÁC SẢN PHẨM {1}

```

ProductController.cs
public class ProductController : Controller
{
    private readonly IProductRepository _productRepository;
    private readonly ICategoryRepository _categoryRepository;

    public ProductController(IProductRepository productRepository,
        ICategoryRepository categoryRepository)
    {
        _productRepository = productRepository;
        _categoryRepository = categoryRepository;
    }

    // Hiển thị danh sách sản phẩm
    public async Task<ActionResult> Index()
    {
        var products = await _productRepository.GetAllAsync();
        return View(products);
    }
}

```

30

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

XEM DANH SÁCH CÁC SẢN PHẨM {2}

```

Index.cshtml
@model IEnumerable<QuanLyBanHang.Models.Product>

@{
    ViewData["Title"] = "Index";
}

<h2>DANH SÁCH SẢN PHẨM</h2>

<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Name)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Price)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Description)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.ImageUrl)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Category)
            </th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>@Html.DisplayFor(modelItem => item.Name)</td>
            <td>@Html.DisplayFor(modelItem => item.Price)</td>
            <td>@Html.DisplayFor(modelItem => item.Description)</td>
            <td>@Html.DisplayFor(modelItem => item.ImageUrl)</td>
            <td>@Html.DisplayFor(modelItem => item.Category.Name)</td>
        </tr>
        <tr>
            <td><a asp-action="Edit" asp-route-id="@item.Id">Sửa</a> |
                <a asp-action="Details" asp-route-id="@item.Id">Chi tiết</a> |
                <a asp-action="Delete" asp-route-id="@item.Id">Xóa</a>
            </td>
        </tr>
    </tbody>
</table>

```

31

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

THÊM MỚI SẢN PHẨM {1}

```

ProductController.cs
// Hiển thị form thêm sản phẩm mới
public async Task<ActionResult> Create()
{
    var categories = await _categoryRepository.GetAllAsync();
    ViewBag.Categories = new SelectList(categories, "Id", "Name");
    return View();
}

// Lưu hình ảnh
private async Task<string> SaveImage(IFormFile image)
{
    var savePath = Path.Combine("wwwroot/images", image.FileName); // Thay đổi đường dẫn theo cấu hình của bạn
    using (var fileStream = new FileStream(savePath, FileMode.Create))
    {
        await image.CopyToAsync(fileStream);
    }
    return "/" + image.FileName; // Trả về đường dẫn tương đối
}

// Xử lý thêm sản phẩm mới
[HttpPost]
public async Task<ActionResult> Create(Product product, IFormFile imageUrl)
{
    if (ModelState.IsValid)
    {
        if (imageUrl != null)
        {
            // Lưu hình ảnh đại diện
            product.ImageUrl = await SaveImage(imageUrl);
        }
        await _productRepository.AddAsync(product);
        return RedirectToAction(nameof(Index));
    }
    // Nếu ModelState không hợp lệ, hiển thị form với dữ liệu đã nhập
    var categories = await _categoryRepository.GetAllAsync();
    ViewBag.Categories = new SelectList(categories, "Id", "Name");
    return View(product);
}

```

32

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

THÊM MỚI SẢN PHẨM {2}

```

Create.cshtml
@model QuanLyBanHang.Models.Product

@{
    ViewData["Title"] = "Create";
}

<h2>Thêm mới sản phẩm</h2>
<div>
    <form asp-action="Create" enctype="multipart/form-data">
        <div class="form-group">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <input type="text" value="Tên mới" class="form-control" />
            </div>
            <div class="form-group">
                <input type="file" asp-for="ImageUrl" class="form-control" />
            </div>
            <div class="form-group">
                <input type="text" value="Giá bán" class="form-control" />
            </div>
            <div class="form-group">
                <input type="text" value="Mô tả" class="form-control" />
            </div>
            <div class="form-group">
                <select asp-for="CategoryId" class="form-control">
                    <option value="0">Chọn danh mục</option>
                </select>
            </div>
            <div class="form-group">
                <input type="submit" value="Thêm mới" class="btn btn-primary" />
            </div>
        </div>
    </form>
</div>
<div class="form-group">
    <input type="text" value="Tên mới" class="form-control" />
</div>
<div class="form-group">
    <input type="file" asp-for="ImageUrl" class="form-control" />
</div>
<div class="form-group">
    <input type="text" value="Giá bán" class="form-control" />
</div>
<div class="form-group">
    <input type="text" value="Mô tả" class="form-control" />
</div>
<div class="form-group">
    <select asp-for="CategoryId" class="form-control">
        <option value="0">Chọn danh mục</option>
    </select>
</div>
<div class="form-group">
    <input type="submit" value="Thêm mới" class="btn btn-primary" />
</div>
</div>

```


33

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

```

// Hình thức form cập nhật sản phẩm
public async Task<ActionResult> Edit(int id)
{
    var product = await _productRepository.GetByIdAsync(id);
    if (product == null)
    {
        return NotFound();
    }

    var categories = await _categoryRepository.GetAllAsync();
    ViewBag.Categories = new SelectList(categories, "Id", "Name", product.CategoryId);
    return View(product);
}

// Xử lý cập nhật sản phẩm
[HttpPost]
public async Task<ActionResult> Edit(Product product, IFormFile imageUrl)
{
    if (ModelState.IsValid)
    {
        if (imageUrl == null)
        {
            // Lưu hình ảnh cũ
            product.ImageUrl = await SaveImage(imageUrl);
        }

        await _productRepository.UpdateAsync(product);

        return RedirectToAction(nameof(Index));
    }
    return View(product);
}

```

CẬP NHẬT SẢN PHẨM {1}

34

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

[illegible]

CẬP NHẬT SẢN PHẨM {2}

35

TAO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

```
ProductController.cs

// Hiển thị thông tin chi tiết sản phẩm
public async Task<IActionResult> Details(int id)
{
    var product = await _productRepository.GetByIdAsync(id);
    if (product == null)
    {
        return NotFound();
    }
    return View(product);
}
```

XEM CHI TIẾT SẢN PHẨM {1}

36

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

```

@model QuanLyBanHang.Models.Product

@{
    ViewData["Title"] = "Details";
}

<div>
    <h4>Chi tiết sản phẩm /h4>
    <hr />
    <div class="row">
        <div class = "col-sm-2">
            @Html.DisplayNameFor(model => model.Name)
        </div>
        <div class = "col-sm-10">
            @Html.DisplayNameFor(model => model.Name)
        </div>
        <div class = "col-sm-2">
            @Html.DisplayNameFor(model => model.Price)
        </div>
        <div class = "col-sm-10">
            @Html.DisplayNameFor(model => model.Price)
        </div>
    </div>
    <div class = "col-sm-10">
        @Html.DisplayNameFor(model => model.Description)
    </div>
    <div class = "col-sm-2">
        @Html.DisplayNameFor(model => model.ImageUrl)
    </div>
    <div class = "col-sm-10">
        
    </div>
    <div class = "col-sm-2">
        @Html.DisplayNameFor(model => model.Category)
    </div>
    <div class = "col-sm-10">
        @Html.DisplayNameFor(model => model.Category.Name)
    </div>
</div>
<div>
    <a asp-action="Edit" asp-route-id="@Model?.Id">Sửa</a> |
    <a asp-action="Index">Trở lại</a>
</div>

```

XEM CHI TIẾT SẢN PHẨM {2}

37

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

```

ProductController.cs
// Hiển thị form xác nhận xóa sản phẩm
public async Task<ActionResult> Delete(int id)
{
    var product = await _productRepository.GetByIdAsync(id);

    if (product == null)
    {
        return NotFound();
    }
    return View(product);
}

// Xử lý xóa sản phẩm
[HttpPost, ActionName("Delete")]
public async Task<ActionResult> DeleteConfirmed(int id)
{
    await _productRepository.DeleteAsync(id);

    return RedirectToAction(nameof(Index));
}

```

XOÁ SẢN PHẨM {1}

38

TẠO CONTROLLER/VIEW CHO CHỨC NĂNG QUẢN LÝ SẢN PHẨM

```

// DeleteConfirmed
[HttpPost, ActionName("Delete")]
public async Task<ActionResult> DeleteConfirmed(int id)
{
    await _productRepository.DeleteAsync(id);

    return RedirectToAction(nameof(Index));
}

// Delete
[HttpGet]
public async Task<ViewResult> Delete(int id)
{
    var product = await _productRepository.GetByIdAsync(id);

    if (product == null)
    {
        return NotFound();
    }

    return View(product);
}

```

```

<div class="row">
    <div class="col-sm-2">
        <h4>Xóa sản phẩm</h4>
        <div class="form-group">
            <input type="text" value="@Model.Id" />
        </div>
        <div class="form-group">
            <input type="button" value="Xóa" class="btn btn-danger" />
        </div>
    </div>
    <div class="col-sm-10">
        <div class="form-group">
            <input type="text" value="@Model.Name" />
        </div>
        <div class="form-group">
            <input type="text" value="@Model.Price" />
        </div>
        <div class="form-group">
            <input type="text" value="@Model.Description" />
        </div>
    </div>
</div>

```

XOÁ SẢN PHẨM {2}

AUTHENTICATION VÀ AUTHORIZATION

Authentication và Authorization

- **Xác thực (Authentication):** Quá trình xác minh danh tính người dùng (họ là ai?).
- **Phân quyền (Authorization):** Quá trình xác định quyền truy cập của người dùng (họ có thể làm gì?).
- Cả hai đều quan trọng để bảo vệ ứng dụng web khỏi truy cập trái phép.


Authentication

Who you are


Authorization

What you can do

41

Authentication

Các phương thức xác thực phổ biến:

- **Cookie-based Authentication:** Dùng trong các ứng dụng web truyền thống.
- **JWT (JSON Web Token):** Dùng cho API.
- **OAuth2/OpenID Connect:** Dùng cho xác thực bên thứ ba (Google, Facebook, Microsoft, v.v.).



42

Authorization

Các cách phân quyền:

- **Phân quyền theo vai trò (Role-based Authorization)**
- **Phân quyền theo chính sách (Policy-based Authorization)**
- **Phân quyền theo quyền hạn (Claims-based Authorization)**

```
[Authorize(Roles = "Admin")]
public class AdminController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

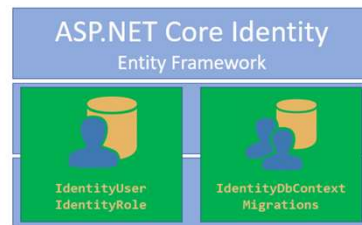
Ví dụ: Phân quyền theo vai trò

43

IDENTITY CORE

Identity Core là gì?

- ASP.NET Core Identity là một framework giúp quản lý người dùng, vai trò, xác thực và phân quyền trong ứng dụng .NET Core.
- Hỗ trợ đăng ký, đăng nhập, xác minh email, đặt lại mật khẩu.
- Có thể mở rộng để thêm thuộc tính tùy chỉnh cho người dùng.



44

CẤU HÌNH IDENTITY CORE

The screenshot shows the 'Manage Packages for Solution' window in Visual Studio. The search bar contains 'Microsoft.AspNetCore.Identity'. The results list several packages, with 'Microsoft.AspNetCore.Identity.EntityFrameworkCore' and 'Microsoft.AspNetCore.Identity.UI' highlighted by a red box. The right pane shows the 'EFCore' package selected, with version 8.0.13 installed.

Package	Version	Installed	Package Level
Project	8.0.13	8.0.13	Top-level
EFCore	8.0.13	8.0.13	Top-level

45

CẤU HÌNH IDENTITY CORE

Models

- ▷ C# ApplicationDbContext.cs
- ▷ C# Category.cs
- ▷ C# ErrorViewModel.cs
- ▷ C# Product.cs
- ▷ C# ProductImage.cs

```

public class ApplicationDbContext : IdentityDbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
        base(options)
    {
    }

    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }
    public DbSet<ProductImage> ProductImages { get; set; }
}

```

Thay thế DbContext thành IdentityDbContext

46

CẤU HÌNH IDENTITY CORE

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddDefaultTokenProviders()
    .AddDefaultUI()
    .AddEntityFrameworkStores<ApplicationDbContext>();

builder.Services.AddRazorPages();

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseStaticFiles();
}

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapRazorPages();

```

Cấu hình IdentityCore (User/Role) vào Program.cs

47

THÊM MIGRATION

```

protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.CreateTable(
        name: "AspNetRoles",
        columns: table => new
        {
            Id = table.Column<string>(type: "nvarchar(450)", nullable: false),
            Name = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
            NormalizedName = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
            ConcurrencyStamp = table.Column<string>(type: "nvarchar(max)", nullable: true)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_AspNetRoles", x => x.Id);
        });

    migrationBuilder.CreateTable(
        name: "AspNetUsers",
        columns: table => new
        {
            Id = table.Column<string>(type: "nvarchar(450)", nullable: false),
            UserName = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
            NormalizedUserName = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
            Email = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
            NormalizedEmail = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true)
        });
}

```

UPDATE DATABASE

```

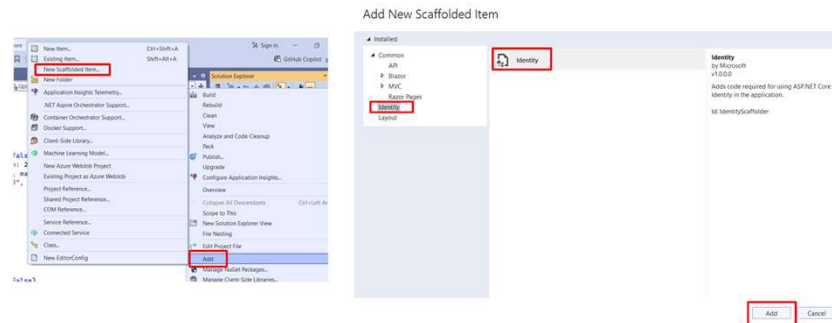
Package Manager Console
Package source: All
PM> Update-Database
Build started...
Build succeeded.
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (360ms) [Parameters=[], CommandType='Text', CommandT
  CREATE DATABASE [SchoolDB];
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (99ms) [Parameters=[], CommandType='Text', CommandT

```

- SchoolDB
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo._EFMigrationsHistory
 - dbo.AspNetRoleClaims
 - dbo.AspNetRoles
 - dbo.AspNetUserClaims
 - dbo.AspNetUserLogins
 - dbo.AspNetUserRoles
 - dbo.AspNetUsers
 - dbo.AspNetUserTokens
 - dbo.Grades
 - dbo.Students

SCAFFOLDING IDENTITY

49



SCAFFOLDING IDENTITY

50

Add Identity

Select an existing layout page, or specify a new one:
(Leave empty if it is in a Razor _viewstart file)

☐ Override all files

Choose files to override:

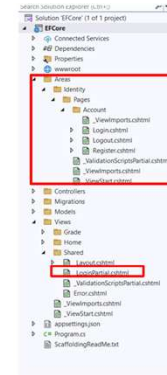
☐ Account.StatusMessage	☐ Account.ExternalLogin	☐ Account.ConfirmEmail
☐ AccountForgotEmailChange	☐ Account.Lockout	☐ AccountForgotPassword
☐ AccountForgotPasswordConfirmation	☐ Account.LoginWithRecoveryCode	☐ Account.Login
☐ Account.LoginWith2fa	☐ AccountManagerLayout	☐ AccountLogout
☐ AccountManagerChangePassword	☐ AccountManagerDeletePersonalData	☐ AccountManagerStatusMessage
☐ AccountManagerDownloadPersonalData	☐ AccountManagerEmail	☐ AccountManagerEnableAuthenticator
☐ AccountManagerPersonalData	☐ AccountManagerGenerateRecoveryCodes	☐ AccountManagerIndex
☐ AccountManagerShowRecoveryCodes	☐ AccountManagerResetAuthenticator	☐ AccountManagerSetPassword
☐ AccountRegisterConfirmation	☐ AccountManagerTwoFactorAuthentication	☐ Account.Register
☐ Account.ResetPasswordConfirmation	☐ Account.ResetEmailConfirmation	☐ Account.ResetPassword

DbContext class: **ApplicationDbContext (EF Core Models)**

Database provider: **Configured from the selected DbContext**

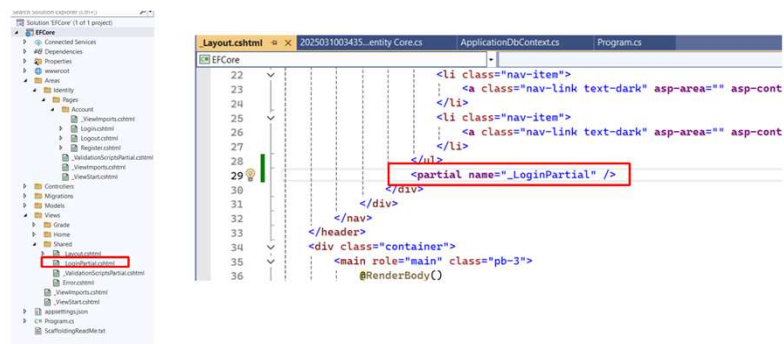
User class:

Chọn các View Identity muốn sinh tự động



THÊM LIÊN KẾT XÁC THỰC VÀO LAYOUT

51



CHẠY CHƯƠNG TRÌNH

52

